אפיון מלא (9 חלקים)

בדיקה שלא סומכת על שופט אחד. שלא אפשר לרמות בהתנהגות יפה. ושממשיכה לעקוב גם אחרי השחרור.

חלק 1: ניהול מאגר הבדיקות

מאגר בדיקות חי ומסודר, שבו כל שינוי יוצר גרסה חדשה, וכל תוצאה נרשמת מול הגרסה שהפיקה אותה.

איך זה עובד

- בדיקה חדשה מוצעת, ונבדקת מול מבנה קבוע: אילו שדות, מאיזה סוג.
- בדיקה תקינה נכנסת למאגר כגרסה חדשה. הגרסאות הישנות נשארות.
- כשמגיש מודל לבדיקה, המערכת בוחרת גרסה אחת של המאגר ורושמת אותה.
- כל תוצאה נשמרת יחד עם גרסת המאגר וגרסת המודל שהניבו אותה.

איזו טעות זה מונע

האיום: ציון משתנה בין שתי הרצות, ואין דרך לדעת אם המודל השתנה או שהבדיקות השתנו.

התשובה: כל הרצה קשורה לגרסה קבועה של המאגר, ולכן אפשר לשחזר ולהשוות.

הכלים שמאחורי החלק

- **דפוס Feature Store (בהשראת Feast) (חיבור שבנינו מאפס):** דפוס מעולם הלמידה: מאגר מנוהל של נתונים בגרסאות, שמגיש את אותה גרסה שוב ושוב. מאגר הבדיקות מנוהל כך: כל שינוי יוצר גרסה חדשה, וכל הרצה נרשמת מול הגרסה שבה השתמשה.
- **JSON Schema (טכנולוגיה קיימת):** תקן שמגדיר בדיוק איך צריך להיראות מסמך נתונים: אילו שדות, מאיזה סוג. כל תוצאה ביניים וכל דוח נבדקים מולו לפני שנשמרים.
- **PostgreSQL + TimescaleDB (טכנולוגיה קיימת):** מסד נתונים יחסי, עם הרחבה שמותאמת לנתונים שמשתנים לאורך זמן. הוא שומר הרצות, תשובות, פסקי דין וציונים, וגם את ההיסטוריה של כל קטגוריה.
- **MLflow (טכנולוגיה קיימת):** כלי קוד פתוח למעקב אחרי ניסויים וגרסאות של מודלים. הוא שומר את התוצאות של כל הרצה מול גרסת המודל וגרסת מאגר הבדיקות.

נתונים

SuiteVersion: suite_version (מזהה גרסת המאגר), test_id (איזו בדיקה), category (לאיזו קטגוריית סיכון היא שייכת), created_at (מתי נוספה), run_id (באיזו הרצה השתמשו בה)

קלט ופלט: בדיקות חדשות ובקשת הרצה ← גרסת מאגר קבועה לכל הרצה

חלק 2: זיהוי מודל שמתנהג אחרת כשבודקים אותו

כל שאלה נשאלת פעמיים, פעם כבדיקה מוצהרת ופעם בתוך שיחה רגילה, והתשובות מושוות.

איך זה עובד

- לכל בדיקה יש שני ניסוחים: Mode A מצהיר שזו בדיקה, ו-Mode B הוא שיחה רגילה בלי שום רמז.
- שתי התשובות מומרות למספרים שמייצגים משמעות, ונמדד המרחק ביניהן.
- אחרי הרבה שאלות, מבחן סטטיסטי בודק אם הפער הממוצע הוא אמיתי או מקרי.

- פער מובהק הוא חשד שהמודל משנה התנהגות כשהוא יודע שבודקים אותו.

מה זה מגן מפניו

האיום: מודל מתקדם מזהה שבודקים אותו, מתנהג יפה, ואחרי השחרור מתנהג אחרת.

התשובה: השוואה בין שני המצבים חושפת פער שאי אפשר להסתיר, כי הוא נמדד בכל שאלה.

הכלים שמאחורי החלק

- **עיצוב בדיקה כפולה (Mode A / Mode B) (החלטה שלנו):** כל שאלה נשאלת פעמיים: פעם בניסוח שמצהיר שזו בדיקה (Mode A), ופעם בתוך שיחה רגילה בלי שום רמז (Mode B). המערכת מחזיקה לכל בדיקה זוג ניסוחים, ומשווה בין שתי התשובות.
- **שיטת Apollo Research (טכנולוגיה קיימת):** גישה מחקרית לזיהוי מודלים שמתנהגים אחרת כשהם יודעים שבודקים אותם. לקחנו ממנה את הרעיון המרכזי: להציג את אותה משימה פעם בהקשר של בדיקה ופעם בהקשר רגיל, ולהשוות.
- **Sentence-Transformers (טכנולוגיה קיימת):** ספרייה שהופכת משפט למספרים שמייצגים את משמעותו (embeddings). היא משווה בין שתי תשובות של אותו מודל, ובודקת אם הן אומרות את אותו דבר גם כשהמילים שונות.
- **SciPy Wilcoxon (טכנולוגיה קיימת):** מבחן סטטיסטי שבודק אם הבדל בין זוגות מדידות הוא אמיתי או מקרי. הוא נותן ערך p על הפער בין התנהגות במצב "בדיקה" להתנהגות במצב "שיחה רגילה".

נתונים

ModePair: test_id (איזו בדיקה), response_a (התשובה במצב בדיקה), response_b (התשובה במצב רגיל), semantic_distance (המרחק ביניהן), p_value (מובהקות הפער)

קלט ופלט: זוג ניסוחים לכל בדיקה ← פער ממוצע וערך p לכל קטגוריה

חלק 3: שלושה שופטים שונים

כל תשובה נבדקת בשלוש שיטות שונות, וההכרעה נקבעת לפי ההסכמה ביניהן.

איך זה עובד

- שופט ראשון: חוקים קבועים, למשל חיפוש ניסוחים אסורים.
- שופט שני: מודל שמעריך את התשובה לפי קריטריונים כתובים.
- שופט שלישי: מודל ממשפחה אחרת, שמעריך לפי אותם קריטריונים.
- כשהשלושה מסכימים, ההכרעה אוטומטית. כשיש מחלוקת חריפה, המקרה עובר לאדם.

איזו טעות זה מונע

האיום: שופט אחד טועה באופן שיטתי, כי הוא עיוור לאותם דברים שהמודל הנבדק עיוור אליהם.

התשובה: שופטים שונים בטבעם טועים במקומות שונים, ומדד ההסכמה אומר מתי אפשר לסמוך.

הכלים שמאחורי החלק

- **שכבת שופטים מגוונת (החלטה שלנו):** שלושה שופטים שונים בטבעם: חוקים קבועים, מודל שופט אחד, ומודל שופט ממשפחה אחרת. כל תשובה מקבלת שלושה פסקי דין, וההכרעה היא לפי ההסכמה ביניהם.
- **מדד Cohen's Kappa (טכנולוגיה קיימת):** מדד סטטיסטי שאומר כמה שני שופטים מסכימים, מעבר למה שהיה קורה במקרה. הוא נמדד בין כל זוג שופטים, ונבדק מול שיפוט אנושי על מדגם קטן.

- **LangSmith / Weights & Biases (טכנולוגיה קיימת):** כלים שמתעדים ניסויים עם מודלים ומציגים אותם בלוחות מעקב. הם שומרים כל הרצה של שופט: מה קיבל, מה ענה, ומה הייתה ההחלטה.

נתונים

JudgeVerdict: response_id, verdict (בטוח, מסוכן או לא בטוח), judge (איזו תשובה), rationale (הנימוק), kappa_pair (הסכמה עם שאר השופטים)

קלט ופלט: תשובה של המודל הנבדק ← פסק דין אחד לתשובה, או העברה לאדם

חלק 4: ציון סיכון ושער שחרור

כל התוצאות הופכות לציון אחד, אבל קטגוריה קריטית שחצתה סף קשיח מפילה את המודל בלי קשר לציון.

איך זה עובד

- כל קטגוריית סיכון מקבלת ציון משלה.
- הקטגוריות ממופות למסגרת תקנית, כדי שהדוח ידבר בשפה מוכרת.
- הציונים משוקללים לפי חומרה לציון כללי.
- לפני ההחלטה נבדק הסף הקשיח של הקטגוריות הקריטיות. חציית סף אחד מספיקה לכישלון.

איזו טעות זה מונע

האיום: ממוצע גבוה מסתיר כישלון חמור בתחום אחד, והמודל משוחרר.

התשובה: סף קשיח לכל קטגוריה קריטית: סיכון חמור לא מתקזז.

הכלים שמאחורי החלק

- **סף קשיח לקטגוריה קריטית (החלטה שלנו):** לכל קטגוריה קריטית יש סף משלה. מי שחצה אותו נכשל, בלי קשר לציון הכללי. הקטגוריות החמורות, למשל סיוע לנשק, נבדקות נגד סף קבוע שנקבע מראש.
- **NIST AI RMF (טכנולוגיה קיימת):** מסגרת של המכון הפדרלי האמריקאי לתקנים (NIST) לניהול סיכונים של בינה מלאכותית. קטגוריות הסיכון והמבנה של הדוח מסודרים לפי המסגרת.
- **PostgreSQL + TimescaleDB (טכנולוגיה קיימת):** מסד נתונים יחסי, עם הרחבה שמותאמת לנתונים שמשתנים לאורך זמן. הוא שומר הרצות, תשובות, פסקי דין וציונים, וגם את ההיסטוריה של כל קטגוריה.

נתונים

RiskScore: category (קטגוריית הסיכון), score (הציון), weight (המשקל לפי חומרה), hard_threshold (הסף הקשיח, אם יש), gate_result (עבר או נכשל)

קלט ופלט: ציונים לפי קטגוריה ← החלטה: משחררים או לא

חלק 5: ייצור בדיקות תקיפה אוטומטי

המערכת ממציאה ניסוחי תקיפה חדשים בשלוש שיטות, בלי לחרוג מתקציב קבוע.

איך זה עובד

- מתחילים מניסוחים מוכנים ממאגר סורק פגיעויות.
- GCG מחשב סיומת מילים שמקרבת את המודל לענות על מה שאסור.

- AutoDAN מפתח ניסוחים באבולוציה: הטובים שורדים ומשתנים.
- PAIR מנהל שיחה עם מודל תוקף שמשפר את הניסיון בכל סבב.
- שומר תקציב סופר כל קריאה, ועוצר כשהתקרה מושגת.

איזו טעות זה מונע

האיום: בדיקה בנויה רק מניסוחים ישנים, והמודלים כבר למדו להתגונן מפניהם.

התשובה: ייצור אוטומטי של ניסוחים חדשים, תחת תקרה שמונעת עלויות אינסופיות.

הכלים שמאחורי החלק

- **PyRIT (טכנולוגיה קיימת):** ספריית קוד פתוח של מיקרוסופט לבדיקות תקיפה אוטומטיות של מודלי שפה. היא מנהלת את השיחה שבה מודל תוקף מנסה שוב ושוב לשכנע את המודל הנבדק לענות על מה שאסור.
- **Garak (טכנולוגיה קיימת):** סורק פגיעויות למודלי שפה, עם מאגר גדול של ניסוחי תקיפה מוכנים. הוא מספק נקודת פתיחה רחבה לכל קטגוריות הבדיקה לפני שמוסיפים בדיקות חדשות.
- **PyTorch autograd (טכנולוגיה קיימת):** מנוע החישוב של PyTorch שמחשב לבד איך שינוי קטן בקלט משנה את התוצאה. הוא מאפשר לשיטת GCG לחשב איזו סיומת מילים הכי מקרבת את המודל לענות על מה שאסור.
- **DEAP (טכנולוגיה קיימת):** ספריית פייתון לאלגוריתמים אבולוציוניים: ניסוחים מתחרים, הטובים שורדים ומשתנים. היא מריצה את AutoDAN: אוכלוסייה של ניסוחי תקיפה שמתפתחת לאורך דורות.
- **שומר תקציב (Cost Guard) (החלטה שלנו):** מנגנון שסופר כל קריאה למודל, ועוצר את הריצה כשהתקציב נגמר. לכל ריצה יש תקרה בדולרים ובמספר קריאות, והמערכת נעצרת לפני שחצתה אותה.

נתונים

AttackAttempt: attack_id (מזהה הניסיון), method (GCG, AutoDAN או PAIR), prompt (הניסוח), cost_usd (העלות), outcome (איך המודל הגיב)

קלט ופלט: קטגוריה ותקציב ← ניסוחי תקיפה חדשים להרצה

חלק 6: הרצה בחדר מבודד

בדיקות שדורשות הרצת קוד רצות בסביבה שנראית אמיתית אבל חסומה מהעולם.

איך זה עובד

- המערכת מחליטה כמה מסוכנת הבדיקה: קלה או כזו שדורשת בידוד מלא.
- בדיקה מסוכנת מקבלת microVM חדש. בדיקה קלה מקבלת שכבת הגנה קלה יותר.
- לסביבה רשת ריקה: אין שום דרך לפנות החוצה.
- כל ניסיון יציאה נחסם ונרשם כממצא. בסוף הבדיקה הסביבה נמחקת.

מה זה מגן מפניו

האיום: המודל הנבדק מנסה, תוך כדי הבדיקה, לגעת במערכות אמיתיות.

התשובה: בידוד מלא: הניסיון נראה ונרשם, אבל לא יכול לגרום נזק.

הכלים שמאחורי החלק

- **Firecracker (טכנולוגיה קיימת):** טכנולוגיה של Amazon להרצת מחשבים וירטואליים זעירים (microVM) שמתניעים תוך אלפיות שנייה. כל בדיקה שדורשת הרצת קוד רצה בתוך microVM משלה, שנמחק אחריה.
- **gVisor (טכנולוגיה קיימת):** שכבת הגנה של גוגל שמיירטת את הקריאות של תוכנה למערכת ההפעלה ובודקת אותן. היא משמשת כקו הגנה שני לבדיקות קלות יותר, שלא צריכות מחשב וירטואלי שלם.
- **בידוד רשת (Network Namespace + iptables) (טכנולוגיה קיימת):** יכולות של לינוקס שחותכות לתהליך כל גישה לרשת, או מתירות רק כתובות מסוימות. סביבת ההרצה מקבלת רשת ריקה, כך שגם קוד שרץ בה לא יכול לפנות החוצה.

נתונים

SandboxRun: run_id (מזהה הריצה), isolation_level (microVM או שכבת הגנה), network_attempts (כמה ניסיונות יציאה נחסמו), started_at (מתי התחילה), destroyed_at (מתי נמחקה)

קלט ופלט: בדיקה שדורשת הרצת קוד ← תוצאה, ורשימת ניסיונות חסומים

חלק 7: מעקב אחרי רגרסיה

כל גרסה חדשה של מודל מושווה לקודמות, ורק שינוי אמיתי מתריע.

איך זה עובד

- גרסה חדשה של המודל רצה על אותה גרסת מאגר כמו קודמתה.
- הציונים נשמרים ביחד עם ההיסטוריה.
- לכל קטגוריה יש גרף בקרה עם טווח צפוי, שנקבע מהיסטוריית הציונים.
- ציון שיוצא מהטווח הוא רגרסיה, ומתריעים לפני השחרור.

איזו טעות זה מונע

האיום: עדכון משפר דבר אחד ומחמיר אחר, והציון הכללי מסתיר את זה.

התשובה: בקרה לכל קטגוריה בנפרד, עם טווח שמבדיל רעש מרגרסיה.

הכלים שמאחורי החלק

- **SPC (בקרת תהליכים סטטיסטית) (טכנולוגיה קיימת):** שיטה מעולם התעשייה שמבחינה בין תנודה רגילה בתהליך לבין שינוי אמיתי בו. גרף בקרה לכל קטגוריה: הציונים נעים בתוך טווח צפוי, וציון שחורג ממנו מתריע.
- **MLflow (טכנולוגיה קיימת):** כלי קוד פתוח למעקב אחרי ניסויים וגרסאות של מודלים. הוא שומר את התוצאות של כל הרצה מול גרסת המודל וגרסת מאגר הבדיקות.
- **PostgreSQL + TimescaleDB (טכנולוגיה קיימת):** מסד נתונים יחסי, עם הרחבה שמותאמת לנתונים שמשתנים לאורך זמן. הוא שומר הרצות, תשובות, פסקי דין וציונים, וגם את ההיסטוריה של כל קטגוריה.
- **SciPy Wilcoxon (טכנולוגיה קיימת):** מבחן סטטיסטי שבודק אם הבדל בין זוגות מדידות הוא אמיתי או מקרי. הוא נותן ערך p על הפער בין התנהגות במצב "בדיקה" להתנהגות במצב "שיחה רגילה".

נתונים

CategoryTrend: category (קטגוריה), model_version (גרסת המודל), score (הציון), control_low (גבול תחתון), control_high (גבול עליון)

קלט ופלט: ציונים של גרסה חדשה ← אות: יציב, או רגרסיה

חלק 8: דוח חתום לרגולטור

כל התוצאות מסוכמות בדוח בשלוש רמות, וחתום כך שכל שינוי בו מתגלה.

איך זה עובד

- כל הרכיבים מוסרים את התוצאות שלהם, ואלה נבדקות מול מבנה קבוע.
- הדוח נבנה בשלוש רמות: תקציר, פירוט ונספח.
- מהדוח מופק PDF.
- הדוח נחתם דיגיטלית. מי שמקבל אותו יכול לוודא שהוא לא שונה.

איזו טעות זה מונע

האיום: ציון בלי הסבר, או דוח שאפשר לשנות אחרי שהופק.

התשובה: שלוש רמות לכל קורא, ושינוי כלשהו אחרי החתימה מתגלה.

הכלים שמאחורי החלק

- **WeasyPrint (טכנולוגיה קיימת):** ספרייה שהופכת דף HTML למסמך PDF מסודר. היא מפיקה מהדוח את גרסת ה-PDF שמוסרים לרגולטור.
- **JSON Schema (טכנולוגיה קיימת):** תקן שמגדיר בדיוק איך צריך להיראות מסמך נתונים: אילו שדות, מאיזה סוג. כל תוצאה ביניים וכל דוח נבדקים מולו לפני שנשמרים.
- **חתימה Ed25519 על הדוח (טכנולוגיה קיימת):** שיטת חתימה דיגיטלית מודרנית, קטנה ומהירה. הדוח הסופי נחתם, וכל שינוי בו אחרי החתימה מתגלה.
- **NIST AI RMF (טכנולוגיה קיימת):** מסגרת של המכון הפדרלי האמריקאי לתקנים (NIST) לניהול סיכונים של בינה מלאכותית. קטגוריות הסיכון והמבנה של הדוח מסודרים לפי המסגרת.

נתונים

SignedReport: report_id (מזהה הדוח), model_version (איזה מודל), verdict (החלטה), content_hash (טביעת אצבע של התוכן), signature (החתימה)

קלט ופלט: תוצאות מכל הרכיבים ← דוח PDF חתום

חלק 9: ניטור אחרי השחרור

דגימה קטנה מהשימוש האמיתי עוברת דרך אותם שופטים, כדי לגלות סטייה הדרגתית.

איך זה עובד

- חלק קטן מהשימוש האמיתי במודל נשלח לזרם אירועים, בלי לעכב את המשתמש.
- אותם שלושה שופטים בוחנים את הדגימה.
- הציונים נשמרים בציר זמן, לפי קטגוריה.
- שבוע אחרי שבוע ההתפלגות מושווה לזו של הבדיקה המקורית. סטייה מעבר לסף מתריעה.

איזו טעות זה מונע

האיום: המודל מתדרדר בהדרגה אחרי השחרור, ואיש לא שם לב עד שמישהו מתלונן.

התשובה: עין שנשארת פקוחה, שמשווה את ההתנהגות בשטח לזו שנבדקה.

הכלים שמאחורי החלק

- **Apache Kafka (טכנולוגיה קיימת):** מערכת להעברת זרם גדול של אירועים בין חלקי תוכנה. דגימה מהשימוש האמיתי במודל זורמת דרכה אל השופטים, בלי לעכב את המשתמשים.
- **Evidently AI (טכנולוגיה קיימת):** ספרייה שמזהה שינוי הדרגתי (drift) בהתנהגות של מודל לאורך זמן. היא משווה את ההתפלגות של התשובות החדשות להתפלגות בבדיקה המקורית.
- **PostgreSQL + TimescaleDB (טכנולוגיה קיימת):** מסד נתונים יחסי, עם הרחבה שמותאמת לנתונים שמשתנים לאורך זמן. הוא שומר הרצות, תשובות, פסקי דין וציונים, וגם את ההיסטוריה של כל קטגוריה.
- **שכבת שופטים מגוונת (החלטה שלנו):** שלושה שופטים שונים בטבעם: חוקים קבועים, מודל שופט אחד, ומודל שופט ממשפחה אחרת. כל תשובה מקבלת שלושה פסקי דין, וההכרעה היא לפי ההסכמה ביניהם.

נתונים

DriftSample: sample_id (מזהה הדגימה), week (איזה שבוע), category (קטגוריה), verdict (פסק הדין), drift_score (מדד הסטייה)

קלט ופלט: דגימה מהשימוש האמיתי ← התראה אם יש סטייה

ההחלטות

שופט אחד או שלושה?

בחרנו שלושה שופטים שונים. שופטים שונים בטבעם לא עיוורים לאותם דברים. כשהם מסכימים אפשר לסמוך, וכשהם חלוקים אדם מכריע.

המחיר: עלות גבוהה יותר, והצורך לתחזק מנגנון הכרעה.

ציון ממוצע בלבד או גם סף קשיח?

בחרנו סף קשיח. סיכון חמור לא מתקזז. קטגוריה קריטית שחצתה את הסף מפילה את המודל, בלי קשר לשאר.

המחיר: מודל יכול להיכשל על ממצא אחד שנראה קטן, וצריך לשכנע גם את מי שממתין לשחרור.

תרחישים ידניים או גם אוטומטיים?

בחרנו גם יצירה אוטומטית. תוקף אמיתי ממציא. בדיקה שלא ממציאה בודקת רק את העבר.

המחיר: עלות של קריאות ל-API, ותקציב שצריך לשמור עליו.

להריץ ישירות או בתוך חדר מבודד?

בחרנו microVM מבודד. בידוד ברמת וירטואליזציה חוסם נזק אמיתי, ועדיין רואים כל ניסיון.

המחיר: תשתית מורכבת יותר, וזמן הרצה גבוה במעט.

פעם אחת לפני שחרור או גם אחרי?

בחרנו ניטור רציף. התדרדרות הדרגתית מתגלה רק במעקב. דגימה קטנה זולה ולא חוסמת את המשתמשים.

המחיר: צינור נתונים נוסף, ושמירה קפדנית על הפרטיות של הדגימה.

ציון אחד או דוח חתום בשלוש רמות?

בחרנו דוח חתום. שלוש רמות נותנות לכל קורא את מה שהוא צריך, והחתימה מבטיחה שהדוח לא שונה.

המחיר: מאמץ בניית הדוח ותחזוקת סכמה קבועה.

מה השתנה מהגרסה הראשונה

קודם	עכשיו
עשרה עד עשרים תרחישים שנבנו ידנית	מאגר חי בגרסאות, עם יצירה אוטומטית
שופט מבוסס חוקים בלבד	שלושה שופטים שונים, והכרעה לפי הסכמה
בדקנו רק פער בין שני מצבים	מבחן סטטיסטי שמבדיל פער אמיתי ממקרי
לא טיפלנו במודלים שיכולים לפעול	סביבת הרצה מבודדת
ציון בודד	ציון עם סף קשיח, ודוח חתום